Assignment 11

Name:Vanga Ranjitth
Hallticket No:2303A51734

Hyperledger Fabric – Comparative Exploration (vs Ethereum)

1 . Objective

To study and practically explore Hyperledger Fabric by setting up a basic Fabric test network, deploying a simple chaincode, and comparing Hyperledger Fabric with a public blockchain (Ethereum) based on architecture, consensus, privacy, performance, and use cases.

2 . Theory

Hyperledger Fabric is an open-source, enterprise-grade permissioned blockchain framework hosted by the Linux Foundation. Unlike public blockchains, it supports pluggable consensus, private channels, and chaincode (smart contracts) written in Go, Java, or JavaScript.

```
# Install Docker sudo apt-get update && sudo apt-get install -y docker.io docker-
compose-plugin sudo usermod -aG docker $USER && newgrp docker

# Install Node.js v18 curl -fsSL
https://deb.nodesource.com/setup_18.x | sudo -E bash sudo aptget
install -y nodejs

# Verify docker --version # Docker
version 24.x node --version # v18.x.x npm
--version # 9.x.x
```

Feature	Hyperledger Fabric	Ethereum (Public)
Type	Permissioned / Private	Permissionless / Public
Consensus	Raft / BFT (pluggable)	Proof-of-Stake (PoS)
Smart Contracts	Chaincode (Go / JS / Java)	Solidity / Vyper
Privacy	Channels + Private Data Coll.	Transparent (public ledger)
TPS	~3,500 TPS	~15-30 TPS
Token Required	No native token	Yes (ETH for gas)
Governance	MSP / Certificate Authority	Token-based governance
Identity	X.509 Certificates	Ethereum address (ECDSA)
Use Cases	Supply Chain, Finance, Healthcare	DeFi, NFTs, DAOs, Gaming
Gas Fees	None	Variable (EIP-1559)
Deployment	Docker containers	Any EVM-compatible node

3. Requirements

OS	Ubuntu 20.04 / Windows WSL / macOS
Docker	v20+; Docker Compose v2+
Git	For cloning fabric-samples repository
cURL	For downloading Fabric binaries
Node.js	v16+ (for JavaScript chaincode & SDK)
RAM	Minimum 8 GB (16 GB recommended)
Fabric Binaries	fabric-samples + bin + config directories

4. Step-by-Step Implementation

Step 1 – Install prerequisites

Step 2 – Download Hyperledger Fabric samples & binaries

```
curl -sSLhttps://bit.ly/2ysbOFE | bash  -s--2.5.01.5.7#
This downloads:
# fabric-samples/ (sample applications) # bin/

(peer, orderer, configtxgen, etc.) # config/

(configtx.yaml, core.yaml, orderer.yaml) export

PATH=$PWD/bin:$PATH export

FABRIC_CFG_PATH=$PWD/config/
```

Step 3 – Start the Fabric Test Network

```
cd fabric-samples/test-network


# Bring down any existing network
./network.sh down


# Start network with Certificate Authority and create a channel ./network.sh up
createChannel -c mychannel -ca
```

```
Creating network "fabric_test" with the default driver
Creating volume "docker_orderer.example.com" ... done
Creating volume "docker_peer0.org1.example.com" ... done
Creating volume "docker_peer0.org2.example.com" ... done
Creating docker_orderer.example.com_1 ... done
Creating docker_peer0.org1.example.com_1 ... done Creating docker_peer0.org2.example.com_1
... done Creating channel 'mychannel'.


Channel 'mychannel' created with height: 1
```

Step 4 – Deploy the basic-asset-transfer chaincode

```
./network.shdeployCC-ccnbasic-ccp../asset-transfer-basic/chaincode-javascript \ -ccl
javascript
```

```
Chaincode is packaged.
Installing chaincode on peer0.org1... Chaincode
installed on peer0.org1.
Installing chaincode on peer0.org2...
Chaincode installed on peer0.org2.
Chaincode definition committed on channel 'mychannel'. Chaincode
definition approved by Org1MSP and Org2MSP.


Committed chaincode definition for chaincode 'basic' on channel 'mychannel': Version:
1.0
Sequence: 1
Validation Plugin: vscc
Endorsement Plugin: escc
```

Step 5 – Set environment variables for Org1 peer CLI

```
exportCORE_PEER_TLS_ENABLED=trueexportCORE_PEER_LOCALMSPID="Org1MSP" export
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org1.example.c
om/peers/peer0.org1.example.com/tls/ca.crt export
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org1.example.com/u
sers/Admin@org1.example.com/msp export CORE_PEER_ADDRESS=localhost:7051
```

Step 6 – Invoke chaincode: Initialise ledger

```
peer chaincode invoke \ -o localhost:7050 \ -ordererTLSHostnameOverride
orderer.example.com \
--tls \ --cafile
"${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example
.com/msp/tlscacerts/tlsca.example.com-cert.pem" \
-C mychannel -n basic \
--peerAddresses localhost:7051 \ --tlsRootCertFiles
"${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.
org1.example.com/tls/ca.crt" \ --peerAddresses localhost:9051 \ --tlsRootCertFiles
"${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.
org2.example.com/tls/ca.crt" \
-c '{"function":"InitLedger","Args":[]}'
```

```
2025-04-12 10:23:14.501 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 001
Chaincode invoke successful. result: status:200
```

Step7–Queryallassets

```
peer chaincode query -C mychannel -n basic \ -
c '{"Args":["GetAllAssets"]}'
```

```
[
{"ID":"asset1","Color":"blue", "Size":5, "Owner":"Tomoko","AppraisedValue":300},
{"ID":"asset2","Color":"red", "Size":5, "Owner":"Brad", "AppraisedValue":400},
{"ID":"asset3","Color":"green","Size":10,"Owner":"Jin Soo","AppraisedValue":500},
{"ID":"asset4","Color":"yellow","Size":10,"Owner":"Max", "AppraisedValue":600},
{"ID":"asset5","Color":"black","Size":15,"Owner":"Adriana","AppraisedValue":700},
{"ID":"asset6","Color":"white","Size":15,"Owner":"Michel","AppraisedValue":800}
]
```

Step8–Transferassetownership

```
peer chaincode invoke \ -o localhost:7050 \ --
ordererTLSHostnameOverride orderer.example.com \
--tls -- cafile "$ORDERER_CA" \
-C mychannel - n basic \
--peerAddresses localhost:7051 - - tlsRootCertFiles "$ORG1_CA" \
--peerAddresses localhost:9051 - - tlsRootCertFiles "$ORG2_CA" \
-c '{"function":"TransferAsset","Args":["asset1","NewOwner"]}'
```

```
2025-04-12 10:25:03.841 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 001
Chaincode invoke successful. result: status:200 payload:"Tomoko"
```

5 . Result & Conclusion

The Hyperledger Fabric test network was successfully set up with two peer organisations (Org1, Org2) and one orderer. The **basic-asset-transfer** JavaScript chaincode was packaged, installed, approved, and committed on channel *mychannel*. Ledger initialisation, querying all assets, and transferring asset ownership were all demonstrated successfully.

The comparative study confirms that Hyperledger Fabric is ideal for enterprise environments where privacy, permissioned access, and regulatory compliance are critical, while Ethereum suits open, decentralised, tokenised ecosystems.